STUDENT INFORMATION MANAGEMENT SYSTEM WITH MOODLE INTEGRATION

Project Progress and Implementation Report

BY

CHARLES HANGOMA 2021414469

CHITUNDU MILIMBO 2020992574

A PROJECT REPORT PRESENTED FOR SUPERVISOR REVIEW TOWARDS THE REQUIREMENTS OF A BACHELOR OF SCIENCE IN COMPUTER SCIENCE DEGREE

THE UNIVERSITY OF ZAMBIA

Department of Computer Science

16 October 2026

COPYRIGHT

We, Charles Hangoma and Chitundu Milimbo, retain our authorship of this project report, subject to the University of Zambia requirements. We provide this copy for academic supervision, source inspection and demonstration.

We retain the notices attached to third-party libraries and our original design sources. Our software dependencies remain subject to their own licences.

DECLARATION

We present this report as our account of the SIS with Moodle integration project and its current implementation. We distinguish implemented source, automated checks, observed demonstrations and work still outstanding. We have not included invented survey responses or institutional approval.

We will each review and sign this declaration before formal submission.

Charles Hangoma

Signature _____ Date _____

Chitundu Milimbo

Signature _____ Date _____

APPROVAL

We submit this checkpoint for supervisor review towards our Bachelor of Science in Computer Science project. We leave approval to the authorised supervisor and department; we do not treat this unsigned page as approval.

Student name _____

Signature _____

Date _____

Supervisor name _____

Signature _____

Date _____

ACKNOWLEDGEMENTS

We acknowledge the Department of Computer Science at the University of Zambia and the technical documentation and open-source tools that support our implementation. We acknowledge our supervisors as the intended reviewers of this checkpoint.

We record named feedback and acceptance only after we receive it. We keep our implementation, testing and review records available so that we can explain our engineering decisions.

DEDICATION

We dedicate our project to the students and university staff whose academic and administrative workflows we aim to support. We want our work to make those workflows clearer, more reliable and easier to complete.

ABSTRACT

We are developing a Student Information System integrated with Moodle to coordinate admissions, registration, academic records, finance, timetabling and student support. We keep formal registration and official academic decisions in the SIS while treating Moodle as a learning-delivery system with separately controlled integration.

We use a handbook-led iterative engineering method, a Next.js web interface, a modular NestJS API and PostgreSQL with Prisma. We implement role and scope checks, immutable decision evidence, idempotent commands and explicit failure states. We use generated participants and a university-like configuration to demonstrate workflows without treating synthetic rules as university policy.

At this checkpoint, we have implemented core applicant, registration, finance, assessment and synthetic timetable journeys. We have improved the staff and student interfaces and direct sign-in to the selected live workspace. We have provisioned a separate MoodleCloud demonstration with four accounts, twelve courses and forty-eight memberships; learner access was confirmed. We have not yet completed automatic registration-driven live Moodle delivery, grade exchange or single sign-on.

We evaluate progress using connected database tests, browser checks, accessibility scans and reproducible local data preparation. We present verified results separately from unfinished live integration and the remaining institutional workflow families.

Keywords: Student Information System, Moodle, registration, academic records, integration, university workflows.

TABLE OF CONTENTS

COPYRIGHT....i
DECLARATION....ii
APPROVAL....iii
ACKNOWLEDGEMENTS....iv
DEDICATION....v
ABSTRACT....vi
TABLE OF CONTENTS....vii
LIST OF TABLES....ix
LIST OF FIGURES....x
LIST OF ABBREVIATIONS....xi
1 INTRODUCTION AND BACKGROUND....1
1.1 Introduction....1
1.2 Background....1
1.3 Statement of the Problem....1
1.4 Aim of the Study....2
1.5 Research Objectives....2
1.6 Research Questions....2
1.7 Significance of the Study....2
1.8 Scope of the Study....2
1.9 Organisation of the Report....3
1.10 Chapter Summary....3
2 LITERATURE REVIEW....4
2.1 Introduction....4
2.2 Background to the Study....4
2.3 Academic Records and Decision Ownership....4
2.4 Moodle Integration and Recovery....4
2.5 Workflow Usability and Accessibility....4
2.6 Related Works and Gaps in the Literature....5
2.7 A Summary of the Related Works....5
2.8 Chapter Summary....5
3 RESEARCH METHODOLOGY....6
3.1 Introduction....6
3.2 Research Design....6
3.3 Research Methods....6
3.4 System Design and Implementation....6
3.4.1 Proposed Business Process....6
3.4.2 Requirements Specification....7
3.4.3 Design Specification....8

3.5 System Implementation 13
3.5.1 System Development 13
3.5.2 System Deployment 13
3.6 Chapter Summary 13
4 RESULTS 14
4.1 Introduction 14
4.2 Baseline Study Results 14
4.3 Analysis of Recorded Evidence 14
4.4 System Automation and Implementation Results 15
4.4.1 System Implementation Results 15
4.4.2 System Validation 18
4.5 Chapter Summary 18
5 DISCUSSION AND CONCLUSIONS 19
5.1 Introduction 19
5.2 Discussion 19
5.3 Conclusions 19
5.4 Recommendations 19
5.5 Future Works 20
5.6 Chapter Summary 20
REFERENCES 21
APPENDICES 22
Appendix A Reproducible review copy 22
Appendix B Engineering diagram source 22
Appendix C Traceability and review 22

LIST OF TABLES

Table 1 Technical source comparison.....5
Table 2 Functional requirements and current evidence.....7
Table 3 Non functional requirements.....7
Table 4 Hardware and software requirements.....13
Table 5 Verification results and boundaries.....14
Table 6 Remaining implementation sequence.....20

LIST OF FIGURES

Figure 1 System context and component architecture 8
Figure 2 Registration and Moodle sequence 9
Figure 3 Domain class model 10
Figure 4 Design class model 11
Figure 5 Timetable publication workflow 12
Figure 6 Student workspace desktop 15
Figure 7 Student timetable desktop 16
Figure 8 Student timetable mobile viewport 17

LIST OF ABBREVIATIONS

Our technical abbreviations

Term	**Meaning**
API	Application Programming Interface
CAT	Central Africa Time
CI	Continuous Integration
DCD	Design Class Diagram
DTO	Data Transfer Object
LMS	Learning Management System
MFA	Multi Factor Authentication
SIS	Student Information System
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modeling Language
UNZA	University of Zambia

1 INTRODUCTION AND BACKGROUND

1.1 Introduction

We are building a workflow-oriented SIS with Moodle integration. In this report we explain the problem, our design method, the implementation we can demonstrate and the work still required. We use our source code and recorded checks to support progress claims [1], [2].

1.2 Background

We treat an SIS as a system of related decisions across a student journey: an application becomes an admission decision, an accepted place becomes a programme attempt, a course plan becomes formal registration, and approved assessment work becomes an official result. We preserve the responsible office, source version and history at each boundary.

We position Moodle as our learning platform. We do not allow an external enrolment delay or a provisional Moodle mark to rewrite the authoritative SIS registration or official record. Our design handbook and approved evidence define these responsibilities [1].

1.3 Statement of the Problem

We need to demonstrate a coherent student journey rather than a collection of disconnected forms. Our main engineering problem is keeping identity, academic context, registration, learning access and official decisions consistent while handling retries, expired authority, corrections and external failures. We also need an interface that supports large staff queues and makes a student’s next action clear.

1.4 Aim of the Study

We aim to design, implement and evaluate a configurable SIS integrated with Moodle that supports university workflows through accountable decisions, usable interfaces and recoverable integration.

1.5 Research Objectives

We model the roles, journeys, rules and data needed for admissions, registration, teaching, results and student services.

We implement connected workflows with live authorisation, audit, state validation and idempotent commands.

We demonstrate the SIS and Moodle boundary using generated accounts and university-like course data.

We evaluate correctness, accessibility and recovery using connected tests, browser checks and supervisor walkthroughs.

1.6 Research Questions

We ask how we can preserve authoritative academic decisions across systems, configure institutional rules without embedding every policy in code, and show staff and students concise, actionable information. We also ask which failure and correction paths we must verify before calling an increment complete.

1.7 Significance of the Study

We intend our work to help students see their current academic position and complete tasks with less uncertainty. We intend staff to work through scoped queues, review batches and preserve evidence instead of relying on ambiguous status text. We provide our supervisors with a reproducible copy so that they can inspect our progress directly.

1.8 Scope of the Study

We demonstrate generated people, public UNZA course-title references and synthetic rules. We do not claim institutional deployment or policy approval. We include admissions, registration, finance, assessment, timetable publication and protected support increments; we keep the remaining v2 families visible in our completion sequence [3].

1.9 Organisation of the Report

We introduce the project in Chapter One, review our technical sources in Chapter Two, explain our method and design in Chapter Three, report implementation and verification in Chapter Four, and discuss conclusions and remaining work in Chapter Five. We include references, diagrams, source links and review instructions in the appendices.

1.10 Chapter Summary

We have defined our aim, scope and evaluation questions. We use the following chapters to connect those objectives to actual implementation evidence.

2 LITERATURE REVIEW

2.1 Introduction

We review the technical and design sources that guide this implementation checkpoint. We do not present this bounded review as the completed scholarly literature chapter: our departmental template requires a broader peer-reviewed review and IEEE references, which we still need to complete before final submission.

2.2 Background to the Study

We organise our review around institutional source ownership, role and scope, reliable integration and workflow usability. Our handbook provides the project-specific evidence and supersession order; our source and task packets show how we translate that design into commands, data and interfaces [1], [2], [4].

2.3 Academic Records and Decision Ownership

We separate proposed plans, formal registrations, provisional assessment data and released results. We use versioned source snapshots so that a later change cannot silently alter the meaning of an earlier decision. We evaluate this separation in the registration and assessment tests rather than assuming that a saved database row is sufficient evidence.

2.4 Moodle Integration and Recovery

We distinguish administrative provisioning from automatic source-driven integration. Our demonstration tooling can create and verify a bounded cloud inventory, but our runtime still needs persisted live-run controls, verified approvals, mappings and durable delivery. We keep this distinction visible in our sequence diagram and progress matrix [5], [6].

2.5 Workflow Usability and Accessibility

We use a persistent workspace shell, concise labels, task panels, readable status summaries and progressive disclosure. We preserve study context and use explicit loading, empty, error and recovery states. We test keyboard access, responsive layout and selected accessibility rules; we still require real user walkthroughs [7], [8].

2.6 Related Works and Gaps in the Literature

We compare the sources available in our project by the decisions they support. We do not fill a ten-article table with invented papers. We will add a verified academic synthesis, search method and traceable peer-reviewed references before formal submission.

Table 1 Technical source comparison

Source	**What we use**	**What it does not prove**
Design handbook [1]	Roles, journeys, policy ownership and approved evidence	Runtime correctness or university adoption
Application design index [2]	Module boundaries and task traceability	Completion of every listed family
Completion sequence [3]	Dependencies and closure criteria	An accepted whole product
Prisma schema and services [4]	Actual models and command boundaries	Human acceptance or deployment health
Moodle demonstration guide [5]	Bounded cloud provisioning and learner verification	Automatic registration or grade exchange
Live foundation amendment [6]	Corrected approval and lifecycle design	A persisted running service
UI task packets [7], [8]	Student shell and exact sign-in direction	Whole-system accessibility certification

2.7 A Summary of the Related Works

We use design sources to define obligations and tests to evaluate our implementation. Our current gap is not an absence of screens alone: we still need several complete owner-to-student workflows and a formal academic literature review. We keep both kinds of work explicit.

2.8 Chapter Summary

We have identified the concepts and evidence that guide our implementation. We next explain how we model, build and verify the system.

3 RESEARCH METHODOLOGY

3.1 Introduction

We use iterative software engineering with traceable vertical increments. We start from a journey and its authority, implement the UI, API, state and audit boundary, and verify normal, refusal, conflict and recovery paths before reviewing the increment.

3.2 Research Design

We conduct a design-and-build project evaluated through technical experiments and planned role walkthroughs. We do not claim a questionnaire sample, hypothesis test or measured user satisfaction at this checkpoint. We keep automated results separate from human evaluation.

3.3 Research Methods

We inspect the controlling handbook, exact approved design evidence, implementation plans and current source. We record missing authority or policy as a gap. We use generated records and explicitly synthetic policy values to exercise workflows, then retain source versions, test output and demonstration receipts.

We divide implementation and review between Charles Hangoma and Chitundu Milimbo through task packets. We preserve each other’s worktrees and request review of concrete changes. We leave the handoff acceptance record unsigned until each developer has actually reviewed the work.

3.4 System Design and Implementation

3.4.1 Proposed Business Process

We guide an applicant from programme discovery through qualifications, supporting documents and submission. We route staff work through assigned admissions queues. We preserve the admission decision before conversion to a programme attempt. We then validate a course plan and issue a formal registration receipt. Teaching, finance, timetable and results views consume the appropriate source-owned records.

3.4.2 Requirements Specification

Table 2 Functional requirements and current evidence

Requirement	**Our implementation**	**Closure boundary**
Identity and workspaces	Password sign-in, live assignments, deliberate switch, safe landing	We deny foreign, expired and emergency defaults.
Admissions	Discovery, qualifications, uploads, assigned review and decisions	We still need broader configured cycles and batch operations.
Registration	Versioned plans, rules, formal receipts and amendments	We still need advanced credit and progression families.
Timetables	Master/course previews, independent publication, owned student views	We still need full recurring teaching and exam operations.
Assessment	Plans, mappings, staging, moderation and official publication increments	We preserve provisional and official boundaries.
Moodle	Simulator and separate cloud provisioning demonstration	We still need automatic live delivery and grade exchange.
Student support	Assigned academic cases, replies, actions and controlled closure	We still need protected specialist services and risk triage.

Table 3 Non functional requirements

Requirement	**How we evaluate it**
Authorisation	We check live role, scope, relationship, state and time on the API.
Integrity and recovery	We use immutable evidence, versions, audit and idempotency.
Usability	We check concise actions, study context, paging and independent scrolling.
Accessibility	We run selected axe checks and keyboard/responsive browser scenarios.
Portability	We provide locked dependencies, migrations, generated data and isolated startup.

3.4.3 Design Specification

We use a modular monolith. Our Next.js interface calls a NestJS API; domain services own commands and PostgreSQL persistence. We isolate external-provider adapters. We use the following source-based engineering diagrams to explain the implemented boundaries and the remaining live integration work.

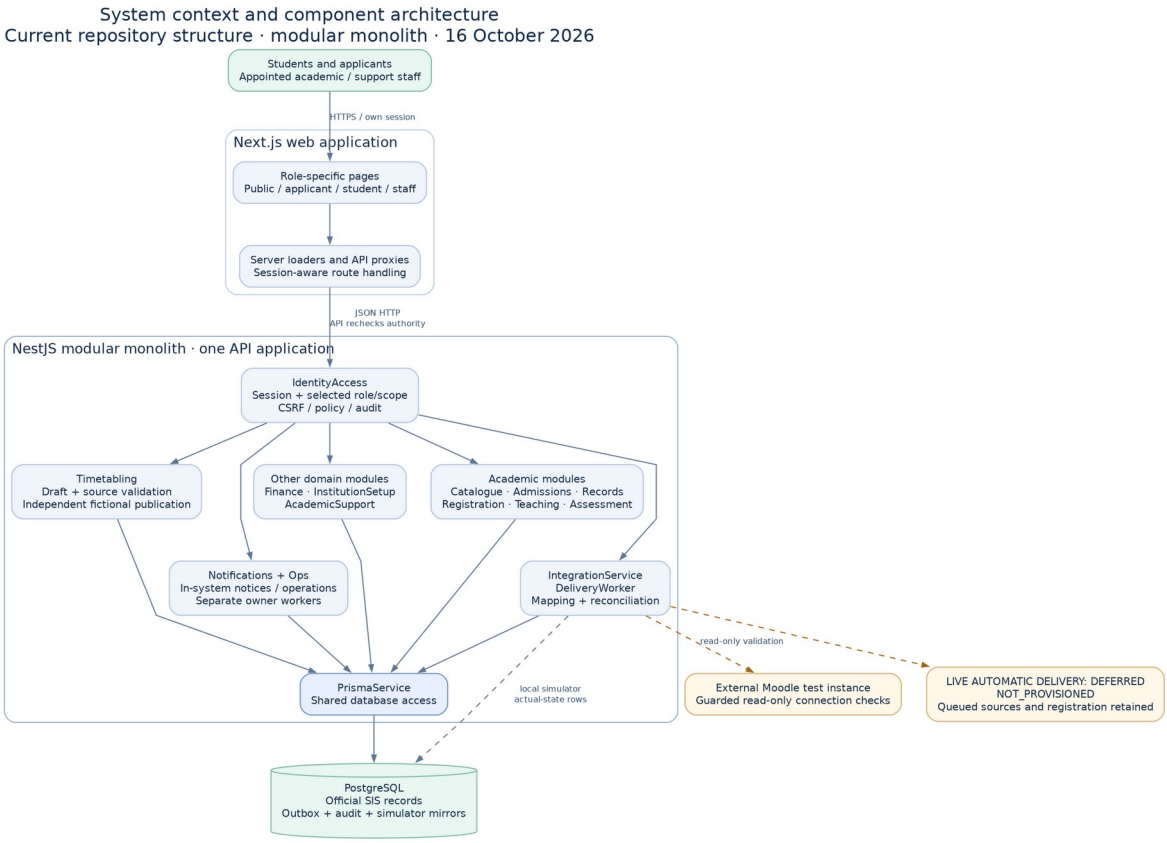


Figure 1 System context and component architecture

We show logical components and ownership rather than claiming that each module is a separately deployed service.

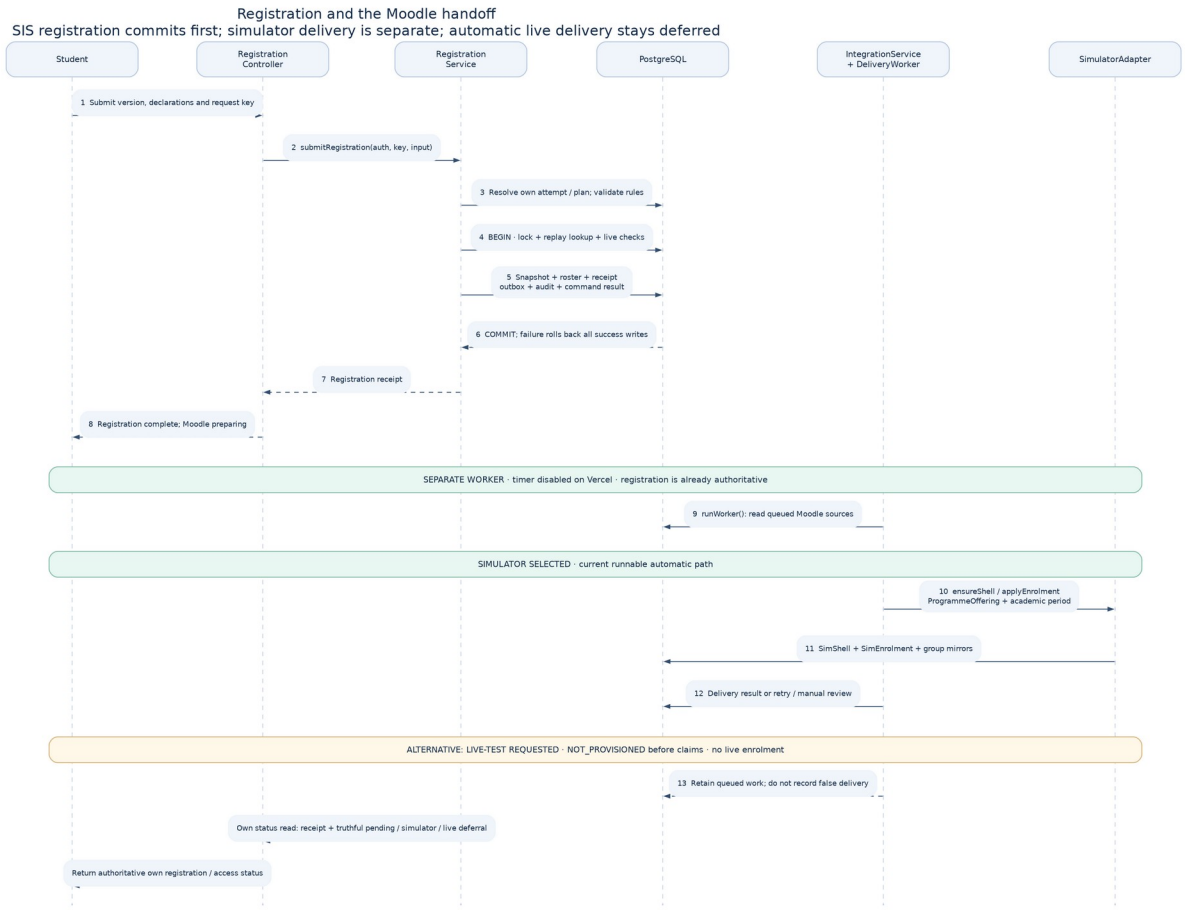


Figure 2 Registration and Moodle sequence

We distinguish our working simulator path and separately provisioned cloud courses from automatic live delivery that remains deferred.

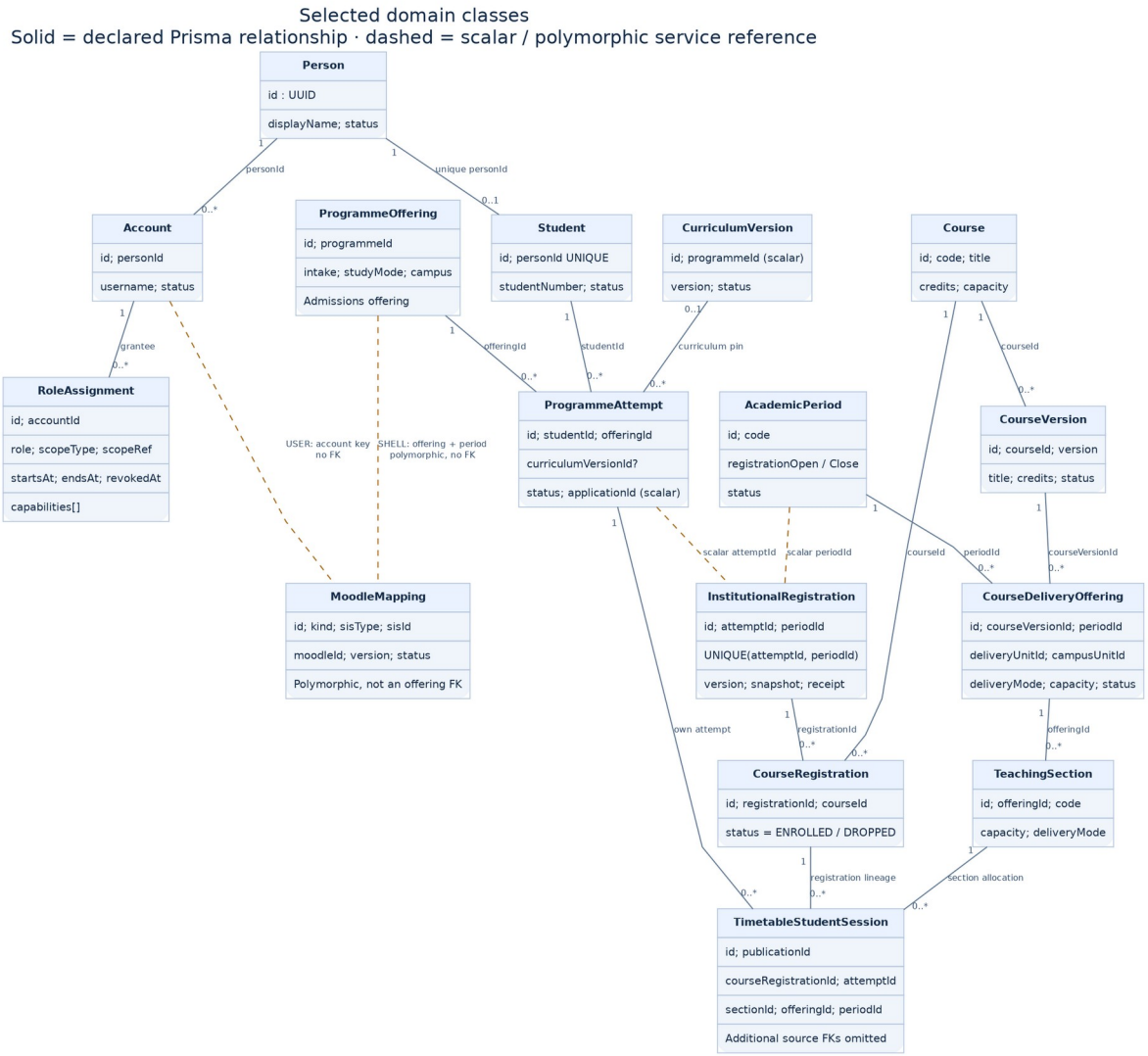


Figure 3 Domain class model

We show a selected model to keep the diagram readable. We retain the complete schema and service implementations in the source package.

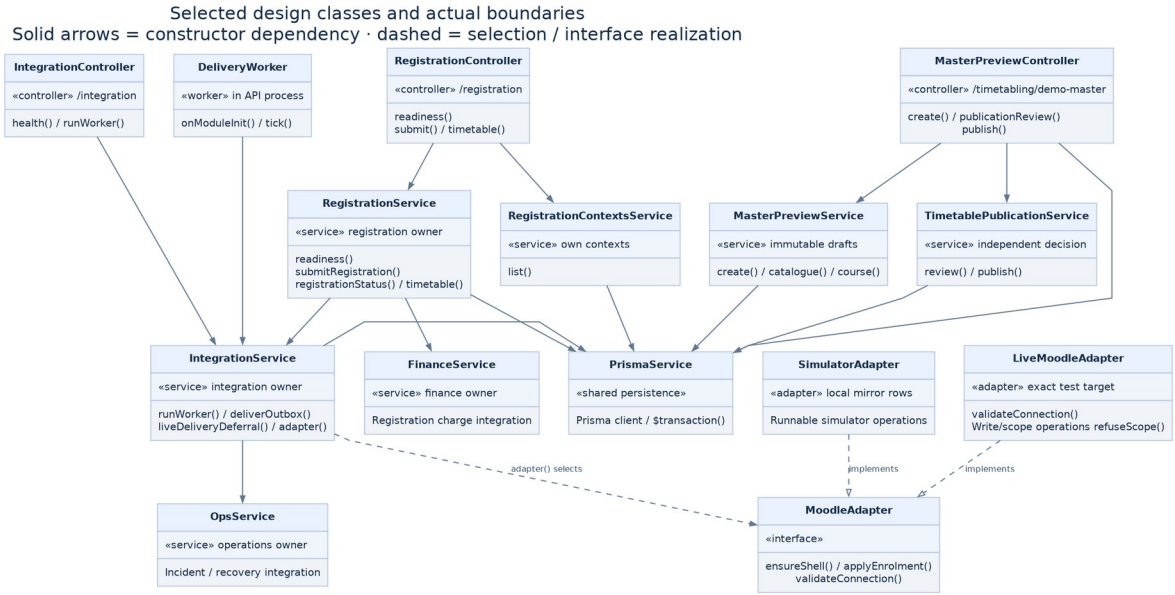


Figure 4 Design class model

We show a selected model to keep the diagram readable. We retain the complete schema and service implementations in the source package.

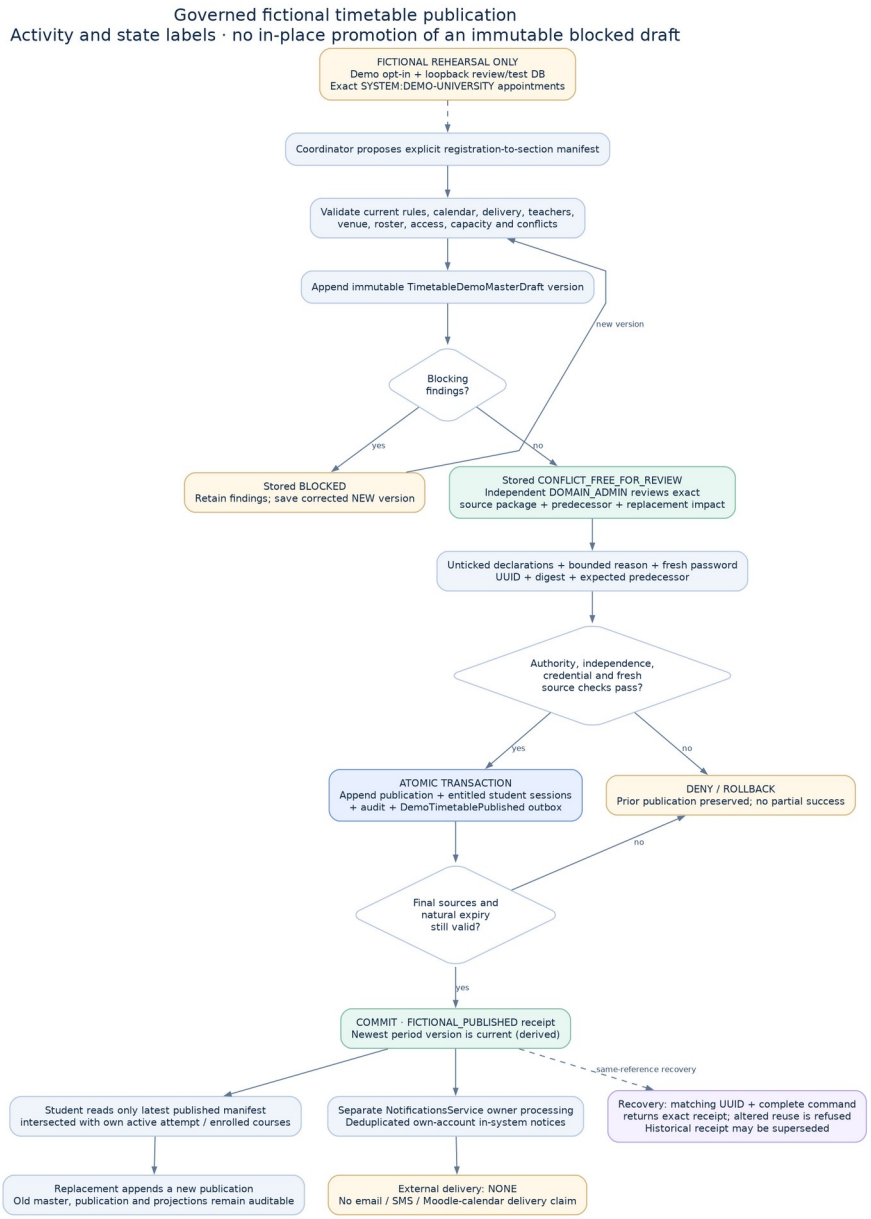


Figure 5 Timetable publication workflow

We require an independent review of the exact timetable source. We preserve registrations when publication or notification delivery changes.

3.5 System Implementation

3.5.1 System Development

We develop in TypeScript, keep contracts and shared UI patterns in packages, and use Prisma migrations for schema changes. We write task packets and learning notes alongside implementation. We protect a partner's active checkout with isolated review worktrees and explicit path staging.

Table 4 Hardware and software requirements

Item	**Our review configuration**
Development tools	Node 24.21.0, npm 11 or later, TypeScript and the committed lockfile
Application stack	Next.js web, NestJS API, PostgreSQL 18 and Prisma
Portable startup	Docker Engine or Desktop with Docker Compose
Review machine	We recommend at least 8 GB RAM and 5 GB free disk space.
Connectivity	We need initial image/package downloads; the local simulator needs no Moodle token.
Client	We use a current browser and test desktop and narrow mobile layouts.

3.5.2 System Deployment

We keep the web and API as separate Vercel deployments and apply database migrations deliberately. Our preceding all-page release was merged and served by both aliases. We treat the new student and sign-in work as a separate checkpoint until its own merge and deployment are verified.

We provide an isolated Docker Compose review copy with local accounts and generated registrations. We exclude cloud tokens, private environment files and deployment databases. We retain setup and workflow instructions in our companion handoff and demonstration guide.

3.6 Chapter Summary

We have connected our requirements to data ownership, service boundaries, diagrams and repeatable evaluation. We next report our observed implementation and verification results.

4 RESULTS

4.1 Introduction

We report current source and recorded execution results. We distinguish earlier merged release evidence, newly verified checks, fixture-driven visual tests and work that has not yet passed a complete connected run.

4.2 Baseline Study Results

We have not conducted a baseline participant study at this checkpoint. We therefore do not report respondents, percentages, statistical significance or interview findings. We use our handbook and technical verification as the current engineering baseline.

4.3 Analysis of Recorded Evidence

We have prepared the same four generated demonstration participants and twelve course titles in a fresh local review database. We have completed nine formal registrations and thirty-six course registrations through normal API commands. We have prepared a separate twelve-session master timetable for independent review; we have not inserted an automatic approval.

Table 5 Verification results and boundaries

Check	**Recorded result**	**Our interpretation**
Preceding merged release	137 main browser; 26 Moodle deferral; 490 API unit; 13 existing skips	We retain these as earlier release evidence, not a blanket pass for this checkpoint.
New auth connected suite	6 of 6 passed on isolated PostgreSQL	We verified selected-context rotation and authority refusal.
Focused workspace and scope units	8 of 8 passed	We verified pure landing/scope behavior.
Config import rerun	267 of 267 passed after build stabilised	We resolved a concurrent clean/build import window.
Student UI fixture suite	39 of 39 passed; five axe scans	We verified layout and interactions with controlled responses.
Fresh local preparation	9 formal registrations; 36 course registrations	We exercised normal API commands and prepared a reviewable timetable.
Full unit invocation on unseeded DB	24 IAM fixture assertions failed	We must rerun the full suite with its required seeded fixture; we do not call this run green.

4.4 System Automation and Implementation Results

4.4.1 System Implementation Results

We have replaced the student top navigation with a persistent side navigation and an account area. We show important study information first, retain programme and period context, and keep the main workspace and navigation scroll independent on desktop.

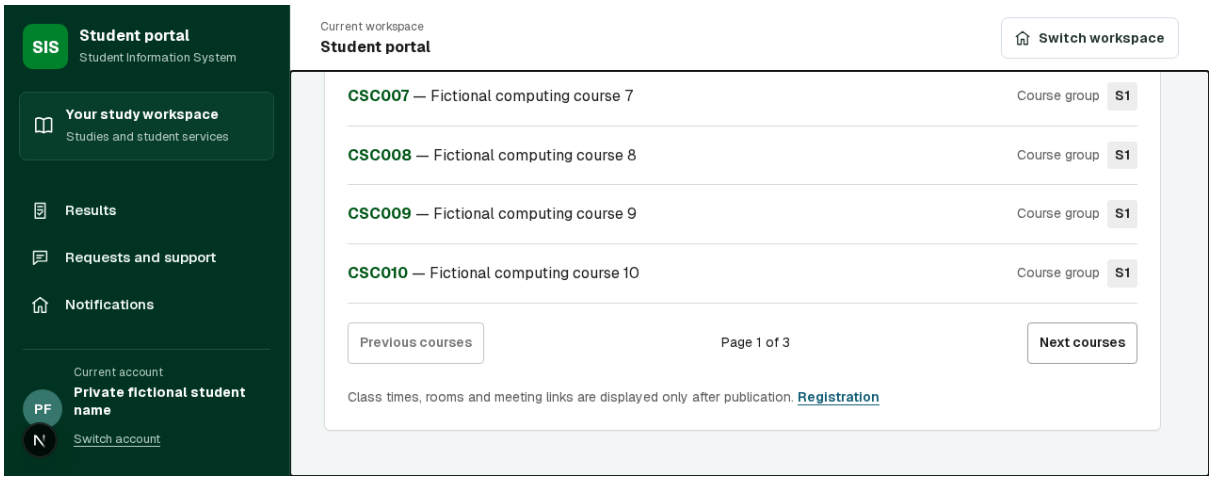


Figure 6 Student workspace desktop from our controlled browser UI test

We identify this image as fixture-driven UI evidence. We do not use it as proof that every displayed backend workflow has completed. We retain actual connected checks separately.

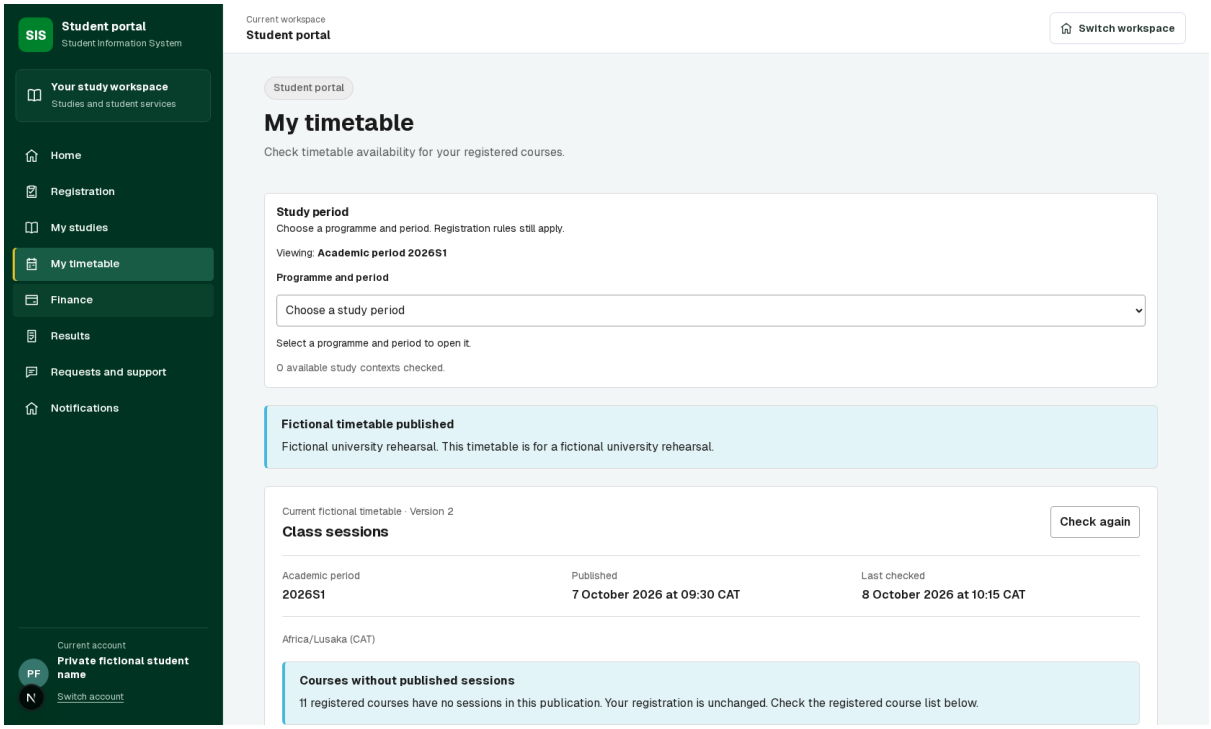


Figure 7 Student timetable desktop from our controlled browser UI test

We present publication status, source version, registered-course coverage and teaching sessions in layers. We retain a clear distinction between a missing timetable and a missing registration.

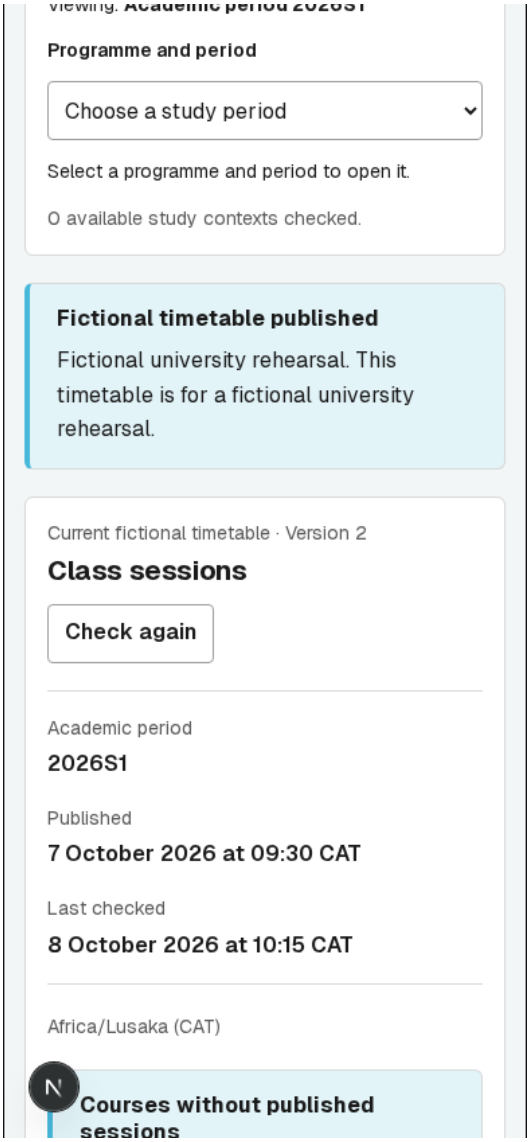


Figure 8 Student timetable mobile viewport from our browser review

We provide a narrow-screen navigation control and timetable alternatives so that a wide weekly grid is not the only way to inspect sessions.

4.4.2 System Validation

We separately provisioned four demonstration Moodle accounts, twelve visible courses and forty-eight memberships. The site owner confirmed that a student could sign in, see the courses and open one. We do not infer automatic SIS-driven enrolment, grade import or single sign-on from that administrative demonstration.

We have built the current web and API source successfully and typechecked the API. We have verified the new auth lifecycle against an isolated migrated database. We still need the final whole-checkpoint browser and seeded unit run, the portable Docker startup check and the fresh deployment check before claiming this increment fully released.

We use failure evidence to improve the verification environment. The full unit run encountered missing seeded IAM fixtures and a concurrent configuration-package clean window; the ten import-failed suites passed after the build stabilised. We retain the unseeded-fixture limitation rather than deleting assertions or reporting a false pass.

4.5 Chapter Summary

We have demonstrated connected core workflows and improved the student interface. We have also identified explicit limits: whole-product completion, the live integration foundation and final human acceptance remain outstanding.

5 DISCUSSION AND CONCLUSIONS

5.1 Introduction

We assess our progress against the project objectives and identify the next complete increments. We avoid a single completion percentage because implemented screens, technical evidence and full institutional workflows are different measures.

5.2 Discussion

We have made our strongest progress where source ownership is explicit: registration commands issue formal receipts, assessment decisions retain their history, and timetable publication reviews a specific source. We still need to extend that discipline across advanced academic configuration and live provider operations.

We have improved convenience through direct workspace landing, scoped queues, reusable task panels and a student sidebar. We still need named walkthroughs to evaluate whether these interfaces support real staff workloads and student tasks without strain.

5.3 Conclusions

We have a substantial runnable demonstration with core SIS workflows and a separately verified MoodleCloud course-access demonstration. We have not completed the whole v2 system or automatic live Moodle delivery. We can use our source package and handoff packet to review the current checkpoint and continue the remaining work in dependency order.

5.4 Recommendations

We recommend closing the current presentation and sign-in increment with complete connected evidence, then completing the persisted Moodle run foundation and registration-driven delivery. We recommend preserving synthetic policy provenance, independent approvals and source-owned academic records throughout that work.

5.5 Future Works

Table 6 Remaining implementation sequence

No.	Our next workflow family	What we require to close it
1	Academic setup and admissions	We need owner-edited curriculum/offering publication and configured workload operations.
2	Registration and progression	We need credit, equivalence, exemption, curriculum change and progression decisions.
3	Teaching and examinations	We need recurring sessions, attendance integrity and complete examination operations.
4	Automatic Moodle delivery	We need persisted runs, fresh approvals, attested mappings, durable claims and reconciliation.
5	Protected student services	We need explainable triage and separate counselling, welfare and safeguarding boundaries.
6	Student AI	We need approved retrieval, owned chats, permissions, citations and refusal/handoff tests.
7	Advanced academic records	We need postgraduate, graduation and issued-credential workflows.
8	Finance and reporting depth	We need provider sandbox evidence and governed exports/metrics.
9	Whole-system acceptance	We need complete journey review, restore/rollback, peak tests and named walkthroughs.

5.6 Chapter Summary

We will use this sequence and the corresponding task packets to close complete workflows. We will update our evidence after each increment and leave acceptance to the named reviewers and supervisors.

REFERENCES

[1] C. Hangoma and C. Milimbo, UNZA SIS Moodle Design Handbook, v3.0.0, controlling project design, including approved evidence and governance registers, repository: unza-sis-moodle-design-handbook-v3.0.0/.

[2] C. Hangoma and C. Milimbo, Application Design Index and Development Instructions, repository: development/DESIGN-INDEX.md and development/AGENTS.md.

[3] C. Hangoma and C. Milimbo, V2 Completion Sequence, 14 October 2026, repository: development/docs/roadmap/V2-COMPLETION-SEQUENCE-2026-10-14.md.

[4] C. Hangoma and C. Milimbo, SIS Data Model and Domain Services, repository: development/prisma/schema.prisma and development/apps/api/src/.

[5] C. Hangoma and C. Milimbo, UNZA Catalogue and Moodle Demonstration, repository: development/docs/demo/UNZA-CATALOGUE-AND-MOODLE-DEMONSTRATION.md.

[6] C. Hangoma and C. Milimbo, Moodle Foundation Amendments and Task Packet, 15 October 2026, repository: development/docs/superpowers/plans/2026-10-15-moodle-foundation-amendments.md and docs/task-packets/TASK-V2-MOODLE-002.md.

[7] C. Hangoma and C. Milimbo, Student Workspace UI 011 Design and Task Packet, repository: development/docs/design/STUDENT-WORKSPACE-UI-011.md and docs/task-packets/TASK-V2-UI-011.md.

[8] C. Hangoma and C. Milimbo, Direct Workspace Landing Task Packet, repository: development/docs/task-packets/TASK-V2-UI-012.md.

[9] University of Zambia, Department of Computer Science, Fourth Year Project Report Template, retained local DOCX and PDF supplied for this project, repository root: report documents/.

APPENDICES

Appendix A Reproducible review copy

We include READ-ME-FIRST.md, compose.supervisor.yml, our dependency lockfile, migrations, generated-data preparation and local review logins. We start the copy with docker compose -f compose.supervisor.yml up --build -d and open localhost:8088 after the web service is ready. We include the detailed workflow steps in our companion guide.

Appendix B Engineering diagram source

We include editable Mermaid source, Graphviz source and rendered SVG/PNG diagrams under development/docs/supervisor/diagrams. We identify the actual schema and service references beside the diagrams. We use rendered images in this report rather than raw diagram code.

Appendix C Traceability and review

We retain task packets UI-011, UI-012 and MOODLE-002, learning notes, the completion sequence and the source manifest. We keep human review fields blank until the relevant developer or supervisor records an actual review. We use our companion Charles and Chitundu handoff packet to exchange implementation and verification duties.